04 — Persistência

Versão 1.0 · atualizado em 08/11/2026

Este documento define como o **Continuum Control** guarda seus dados em três mecanismos (PostgreSQL, MongoDB e Redis), por que cada um foi escolhido e como eles se mantêm consistentes entre si. Ele responde ao §20 e ao §21 do enunciado. As referências `D` e `A` continuam a numeração dos docs 01 a 03.

Arquivos	`V1__schema_inicial.sql` e `V2__permissoes_aplicacao.sql` (PostgreSQL) · `mongo/ocorrencias-schema.json` (validador do MongoDB) · `docker-compose.yml` e `docker/` (ambiente) · `.env.example` (configuração)
Verificação	`verificacao/testes-esquema.py` (46 casos) e `verificacao/testes-persistencia.py` (26 casos), contra os três bancos reais (seção 9)
Documentos relacionados	00 — Enunciado · 01 — Modelo de domínio · 02 — Modelo de dados · 03 — Estados · 05 — API · 06 — Segurança

Sumário

1. Visão geral
2. Por que três mecanismos
3. PostgreSQL
4. MongoDB
5. Redis
6. Consistência entre os bancos: outbox
7. Configuração
8. Execução
9. Verificação
10. Decisões de modelagem
11. Pontos em aberto
12. Histórico de revisões

1. Visão geral

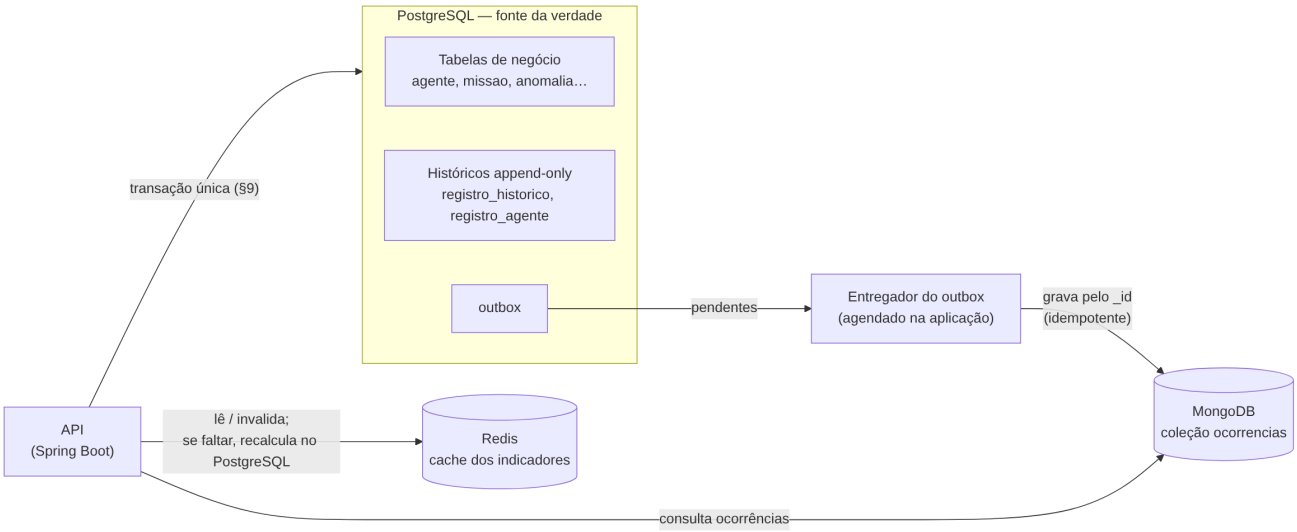


Diagrama do documento (bloco Mermaid)

Mecanismo	Modelo	Guarda	Quem grava	Quem lê
PostgreSQL 16	Relacional	Tudo que tem regra de integridade ou entra numa transação: agentes, eventos, anomalias, missões, designações, autorizações, intervenções, encerramentos, históricos, usuários e o *outbox*	A API, sempre numa transação	A API
MongoDB 7	Documento	As ocorrências (§11), com estrutura diferente por tipo	Só o entregador do *outbox*	A API (consultas de ocorrências)
Redis 7	Chave-valor	O Estado Operacional e o Índice de Estabilidade já calculados (§14, §15)	A API, ao calcular	A API

O PostgreSQL é a **fonte da verdade**. O MongoDB recebe as ocorrências a partir dele (seção 6). O Redis só guarda cópias que podem ser apagadas a qualquer momento: perder o Redis inteiro não perde nenhum dado.

2. Por que três mecanismos

O §20 pede, para cada mecanismo, cinco respostas. Elas estão abaixo, seguidas do que se perderia guardando tudo num só.

2.1 PostgreSQL (relacional)

Pergunta do §20	Resposta
Que problema resolve	Manter coerentes entidades muito ligadas entre si: uma missão não inicia sem agente disponível, um agente não está em duas missões, uma anomalia irreversível não recebe correção (§7, §9)
O que guarda	As 11 tabelas de negócio e de histórico do doc 02, mais o *outbox*
Por que o modelo é adequado	Chaves estrangeiras, `UNIQUE`, `CHECK` e transações ACID garantem no próprio banco boa parte das regras (doc 02, seção 7). As operações compostas do doc 03 alteram três ou quatro tabelas como uma unidade
Característica da aplicação	Quase toda operação relevante muda várias entidades de uma vez e precisa de "tudo ou nada" (§9, §20, §21)

Pergunta do §20	Resposta
Por que não usar só ele	Daria para guardar as ocorrências em `JSONB` (D41), mas o §20 exige um NoSQL de modelo não chave-valor. E calcular o índice a cada requisição repetiria consultas de contagem sobre várias tabelas

2.2 MongoDB (documento)

Pergunta do §20	Resposta
Que problema resolve	Guardar ocorrências de seis tipos com campos diferentes (§11: "nem todas as ocorrências possuirão a mesma estrutura") sem uma tabela por tipo nem colunas vazias
O que guarda	A coleção `ocorrencias`: um documento por ocorrência, com campos comuns e os campos do seu tipo (seção 4)
Por que o modelo é adequado	Cada documento tem a forma do seu tipo. O validador `$jsonSchema` confere os campos de cada tipo no próprio banco, e um tipo novo não exige migração
Característica da aplicação	As ocorrências são registros de campo: muitos por missão, nunca alterados, consultados por missão, agente ou tipo, e sem regra que dependa de outras tabelas além da participação na missão
Por que não guardar no relacional	Herança no relacional exige uma tabela por subtipo (com junções) ou uma tabela única cheia de colunas nulas. A alternativa `JSONB` é discutida em D41

2.3 Redis (chave-valor)

Pergunta do §20	Resposta
Que problema resolve	O Estado Operacional e o Índice de Estabilidade (§14, §15) dependem de contagens sobre agentes, anomalias, missões e intervenções. São lidos com frequência (painel) e mudam raramente em comparação
O que guarda	Uma chave com o resultado do cálculo, com prazo de validade (seção 5)
Por que o modelo é adequado	Leitura e escrita por chave em memória, com expiração nativa (TTL). O valor é um resultado pronto, sem nenhuma consulta sobre ele
Característica da aplicação	Um valor derivado, caro de calcular, que pode estar alguns segundos desatualizado sem prejuízo e que deve ser igual para todas as instâncias da API
Por que não usar os outros	No PostgreSQL seria uma tabela de cache com limpeza manual e sem TTL. Um cache em memória da própria aplicação (Caffeine) não seria compartilhado entre instâncias, e cada uma invalidaria só o seu

3. PostgreSQL

O modelo de dados está no doc 02. Aqui ficam o uso das transações e os usuários de banco.

3.1 Transações

- Cada caso de uso de serviço é um método `@Transactional`. As operações compostas do doc 03 (seção 6) são exatamente esses métodos.
- Isolamento `READ COMMITTED` (o padrão do PostgreSQL). As corridas relevantes são barradas por `UNIQUE` e por `@Version` em `Agente` (doc 02, seção 7.3), sem precisar de `SERIALIZABLE`.
- `spring.jpa.open-in-view=false`: nenhuma consulta acontece fora da transação do serviço.
- `ddl-auto=validate`: o Hibernate confere o mapeamento contra o esquema, mas quem cria o esquema é o Flyway.

3.2 Dois usuários de banco (D43)

Usuário	Usado por	Permissões
`continuum_migracao`	Flyway	Dono do banco e de todas as tabelas: cria e altera o esquema
`continuum_app`	Aplicação	`SELECT`, `INSERT`, `UPDATE`, `DELETE` nas tabelas de negócio; só `SELECT` / `INSERT` em `registro_historico` e `registro_agente`; nada no histórico do Flyway; nenhum DDL

- Os dois são criados por `docker/postgres/init/01-usuarios.sh` na primeira subida do contêiner.
- As permissões da aplicação são dadas pela migração `V2`, que recebe o nome do usuário pelo *placeholder* `${appUser}` do Flyway.
- **Num ambiente de um usuário só** (testes com Testcontainers), o V2 detecta que `appUser` é o próprio usuário do Flyway e não faz nada. Os *triggers* append-only continuam valendo.
- Para a aplicação, a inserção em colunas `GENERATED ALWAYS AS IDENTITY` não exige permissão na sequência (verificado no caso V1 da suíte).

4. MongoDB

4.1 A coleção `ocorrencias`

Um documento por ocorrência. Os campos comuns vêm da classe abstrata `Ocorrencia` (doc 01); os demais, da subclasse indicada por `tipo`.

```
{
  "_id": { "$oid": "6730f2a1c0ffee0000000042" },
  "tipo": "DISTORCAO_TEMPORAL",
  "missaoId": { "$numberLong": "7" },
  "agenteId": { "$numberLong": "3" },
  "dataHora": { "$date": "2026-11-08T21:14:00Z" },
  "observacao": "Oscilação percebida ao atravessar o fórum",
  "intensidade": 7,
  "duracao": { "$numberLong": "2700" }
}
```

Campo	Tipo BSON	Obrigatório	Observação
`_id`	`objectId`	sim	Gerado **na aplicação**, antes de gravar o *outbox* (seção 6)
`tipo`	`string`	sim	Valor de `TipoOcorrencia`; também é o discriminador da herança (seção 4.3)
`missaoId`, `agenteId`	`long`	sim	Referências lógicas ao PostgreSQL, sem FK (doc 02, D21)
`dataHora`	`date`	sim	Instante do registro, em UTC
`observacao`	`string`	sim	Até 4.000 caracteres

Campos de cada tipo (os de `DistorcaoTemporal` e `EvidenciaTemporal` vêm do §11; os demais foram propostos no doc 01, D4):

`tipo`	Campos	Regras do validador
`OBSERVACAO_HISTORICA`	`fatoObservado` (string), `conformeEsperado` (bool)	ambos obrigatórios
`ALTERACAO_DETECTADA`	`valorEsperado`, `valorObservado` (string)	ambos obrigatórios
`EVIDENCIA_TEMPORAL`	`objeto`, `periodoEstimado` (string), `grauConfiabilidade` (double)	`grauConfiabilidade` entre 0 e 1

tipo	Campos	Regras do validador
`DISTORCAO_TEMPORAL`	`intensidade` (int), `duracao` (long, **segundos**)	`intensidade` de 1 a 10; `duracao` ≥ 0
`ENCONTRO_INESPERADO`	`envolvidos` (string), `houveContato` (bool)	ambos obrigatórios
`INCIDENTE_OPERACIONAL`	`gravidade` (valor de `Impacto`), `medidaTomada` (string)	`gravidade` dentro do enum

As faixas de `intensidade` (1 a 10) e de `grauConfiabilidade` (0 a 1) não estão no enunciado; foram definidas aqui para que a validação tenha limites.

4.2 Validador

`ocorrencias-schema.json` é um `$jsonSchema` com os campos comuns e um `oneOf` com um ramo por tipo. Cada ramo fixa o `tipo` e exige os seus campos, então um documento de distorção com campos de observação é recusado. Campos extras são permitidos, para que um campo novo não quebre documentos antigos.

O validador roda com `validationLevel: strict` e `validationAction: error`. Ele é a mesma defesa que o `CHECK` é no PostgreSQL: a validação principal continua no DTO (doc 01, D32).

4.3 Mapeamento em Java (Spring Data MongoDB)

- `Ocorrencia` é `@Document("ocorrencias")`, abstrata, e cada subclasse leva `@TypeAlias` com o valor do enum (ex.: `@TypeAlias("DISTORCAO_TEMPORAL")`).
- O conversor usa `tipo` como chave de tipo (`new DefaultMongoTypeMapper("tipo")`) em vez de `_class`. Assim o campo que o validador confere é o mesmo que o Spring usa para instanciar a subclasse certa, e o documento não carrega nomes de classe Java.
- `Duration` não tem conversão padrão: registrar em `MongoCustomConversions` um par de conversores `Duration` ↔ `Long` (segundos).
- `LocalDateTime` é gravado como `date` em UTC, coerente com `hibernate.jdbc.time_zone=UTC` (doc 02, D22).

Esses quatro pontos precisam de um teste de integração que grave e leia de volta um documento de cada tipo (seção 9.3).

4.4 Índices

Índice	Campos	Consulta atendida
`ix_ocorrencias_missao`	`missaoId`, `dataHora`	Ocorrências de uma missão, em ordem (doc 05)
`ix_ocorrencias_agente`	`agenteId`, `dataHora`	Ocorrências registradas por um agente
`ix_ocorrencias_tipo`	`tipo`	Filtro por tipo

4.5 Criação na subida (D44)

A aplicação cria a coleção ao subir, se ela não existir, com o validador e os índices. Se já existir, atualiza o validador com `collMod`. A operação é idempotente (caso MG0 da seção 9), então vale para qualquer ambiente, inclusive os testes com Testcontainers.

O usuário da aplicação (`readWrite` só no banco `continuum`) é criado por `docker/mongo/init/01-usuario.js` na primeira subida do contêiner.

5. Redis

5.1 O que fica no cache

Chave	Conteúdo	Validade
`continuum:indicadores::atual`	JSON com o Estado Operacional completo, que inclui o Índice de Estabilidade e o estado geral (doc 01, D36), e o instante do cálculo (`calculadoEm`)	60 s (configurável)

Os dois endpoints de indicadores (doc 05) leem a mesma chave: um cálculo atende os dois.

5.2 Leitura e invalidação (D42)

- **Leitura (*cache-aside*):** lê a chave; se não existir, calcula no PostgreSQL e grava com a validade. Em Spring: `@Cacheable(value = "indicadores", key = "'atual'")` no serviço de indicadores, com o prefixo `continuum:` configurado no `RedisCacheConfiguration` (`prefixCacheNameWith`).
- **Invalidação depois do *commit*:** todo caso de uso que muda algo contado no índice apaga a chave quando a transação confirma: situação de agente, situação ou risco de anomalia, situação de missão e intervenção `SEVERO`. Em Spring: `@CacheEvict(value = "indicadores", allEntries = true)` nesses serviços, com o `RedisCacheManager` criado com `.transactionAware()`, que adia o `evict` para depois do *commit*.
- **Por que depois do *commit*:** se a chave fosse apagada antes, uma leitura simultânea poderia recalcular com os dados antigos e gravá-los de novo. Mesmo depois do *commit* sobra uma janela pequena para isso; a validade de 60 s limita quanto tempo um valor antigo sobrevive.
- **Redis fora do ar:** um `CacheErrorHandler` que só registra o erro (`LoggingCacheErrorHandler`) faz a requisição seguir calculando direto no PostgreSQL. O cache nunca derruba a API (caso RD3).

5.3 Configuração do servidor

Sem persistência em disco (`--save ""`, `--appendonly no`), com limite de memória e descarte das chaves menos usadas (`allkeys-lru`). É um cache: perder o conteúdo só custa um recálculo.

6. Consistência entre os bancos: outbox

6.1 O problema (A14)

Registrar uma ocorrência grava em dois lugares: a ocorrência no MongoDB e o registro `OCORRENCIA` no histórico do PostgreSQL (§12). Os dois bancos não compartilham transação. Gravando um depois do outro, uma falha entre as duas gravações deixaria uma ocorrência sem histórico ou um histórico apontando para uma ocorrência que não existe (§21).

6.2 A solução (D40)

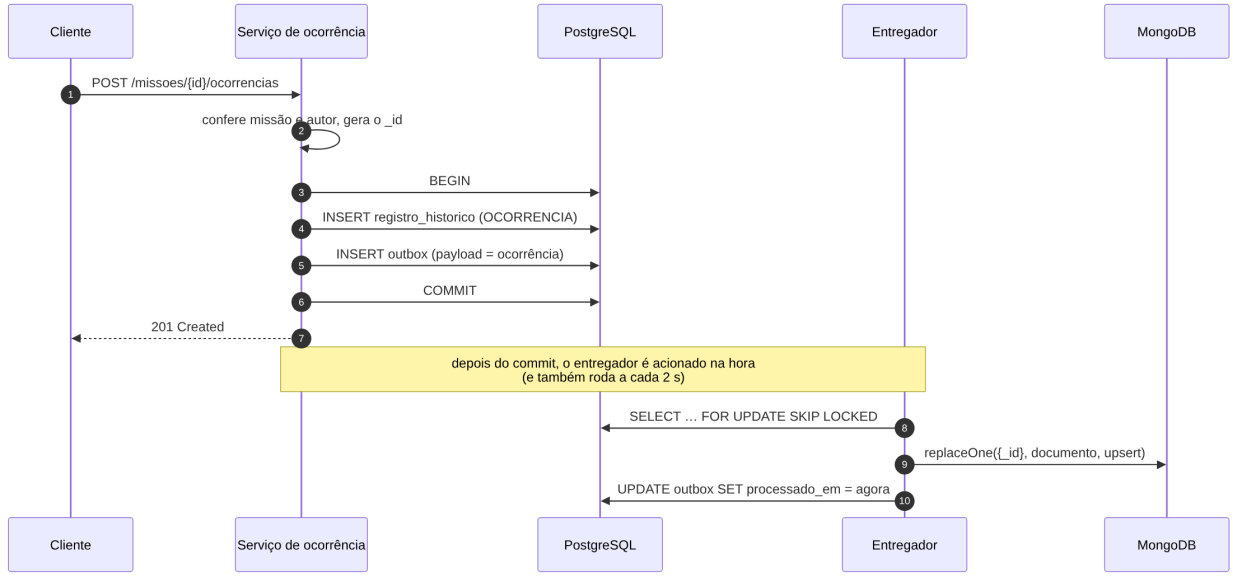


Diagrama do documento (bloco Mermaid)

1. O caso de uso grava o histórico e a ocorrência inteira (`payload`, em JSON) na tabela `outbox`, **na mesma transação**. Ou as duas gravações acontecem, ou nenhuma.
2. Depois do *commit*, o entregador leva cada linha pendente ao MongoDB e marca a linha como processada.
3. O entregador é acionado logo após o *commit* (`@TransactionalEventListener(phase = AFTER_COMMIT)`), então no caso normal a ocorrência já está no MongoDB quando a resposta chega ao cliente. Ele também roda a cada 2 s (`@Scheduled`) para pegar o que ficou pendente.

6.3 Regras do entregador

Regra	Por quê
Uma transação do PostgreSQL por linha, com `SELECT … FOR UPDATE SKIP LOCKED`	Várias instâncias (ou execuções) podem entregar ao mesmo tempo sem pegar a mesma linha (caso OB6)
Gravação no MongoDB pelo `_id`, com *upsert* (`MongoTemplate.save`)	Entregar duas vezes a mesma ocorrência dá no mesmo documento. Isso cobre a queda entre a gravação no MongoDB e a marcação no PostgreSQL (caso OB4)
Só linhas com `proxima_tentativa` ≤ agora, em ordem de `proxima_tentativa`	Uma linha que falhou espera a sua vez sem travar as outras
Falha do documento (o validador recusou): `tentativas + 1`, `ultimo_erro`, `proxima_tentativa` = agora + 2^tentativas s (máximo 5 min), e segue para a próxima	Um documento com defeito não bloqueia a fila (caso OB5)
MongoDB indisponível: adia a linha do mesmo jeito e **encerra o ciclo**	Se o MongoDB está fora, as outras linhas também falhariam (caso OB3a)
Nenhuma linha é abandonada	Uma queda longa do MongoDB só atrasa as entregas (casos OB3b e OB3c)

A revisão encontrou um defeito no primeiro desenho: com um limite fixo de tentativas e sem espera entre elas, uma queda de alguns segundos do MongoDB fazia o entregador esgotar as tentativas e abandonar as ocorrências. A coluna `proxima_tentativa` e a regra de encerrar o ciclo corrigiram isso, e o caso OB3 garante que o defeito não volte.

6.4 O que o cliente vê

- `POST` responde `201 Created` com a ocorrência montada pela aplicação. O registro já é durável, porque está no *outbox*.
- `GET` de ocorrências lê o MongoDB. No caso normal a ocorrência já foi entregue. **Enquanto o MongoDB estiver fora**, as ocorrências novas não aparecem nas consultas, mas o registro `OCORRENCIA` aparece no histórico da anomalia, com o `_id` em `valor_novo`.

6.5 Monitoramento e limpeza

- Um `HealthIndicator` do Actuator, `outbox`, informa quantas linhas estão pendentes e há quanto tempo está a mais antiga. Se a mais antiga passar de 5 minutos, o status vira `ATRASADO`, mapeado para HTTP 200 para não derrubar o contêiner, e a aplicação registra um `WARN`.
- Um job diário apaga as linhas processadas há mais de 7 dias (A15).

---

7. Configuração

Nada que dependa do ambiente fica no código (§24). Tudo vem de variáveis de ambiente, com o `.env` local fora do repositório.

Variável	Uso	Exemplo
`DB_URL`	JDBC do PostgreSQL	`jdbc:postgresql://localhost:5432/continuum`
`DB_APP_USER` / `DB_APP_PASSWORD`	Usuário da aplicação	`continuum_app`
`DB_MIGRACAO_USER` / `DB_MIGRACAO_PASSWORD`	Usuário do Flyway	`continuum_migracao`
`MONGO_URI`	Conexão do MongoDB, com usuário e banco	`mongodb://continuum_app:…@localhost:27017/continuum?authSource=continuum`
`REDIS_HOST` / `REDIS_PORT` / `REDIS_PASSWORD`	Conexão do Redis	`localhost` / `6379`
`JWT_SECRET`	Segredo de assinatura dos tokens (doc 06)	32 bytes aleatórios ou mais
`SPRING_PROFILES_ACTIVE`	Perfil (`dev`, `test`, `prod`)	`dev`

Trecho de referência do `application.yml`:

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_APP_USER}
    password: ${DB_APP_PASSWORD}
  flyway:
    user: ${DB_MIGRACAO_USER}            # o Flyway conecta com o dono do esquema
    password: ${DB_MIGRACAO_PASSWORD}
    placeholders:
      appUser: ${DB_APP_USER}            # usado no V2__permissoes_aplicacao.sql
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate
    properties.hibernate.jdbc.time_zone: UTC
  data:
    mongodb.uri: ${MONGO_URI}
    redis:
      host: ${REDIS_HOST}
```

```
      port: ${REDIS_PORT:6379}
      password: ${REDIS_PASSWORD:}
  cache.type: redis

continuum:
  cache.indicadores.validade: 60s
  indice:                                  # doc 01, D36
    janela: 30d
    penalidade-risco: { BAIXO: 2, MODERADO: 5, ALTO: 10, CRITICO: 20 }
    penalidade-missao-em-execucao: 2
    penalidade-missao-falhou: 8
    penalidade-intervencao-severa: 5
  outbox:
    intervalo: 2s
    espera-maxima: 5m
    alerta-pendente-apos: 5m
    retencao-processados: 7d
  jwt:                                     # doc 06
    segredo: ${JWT_SECRET}
    validade: 60m
```

- Com Spring Boot 3 e Flyway 10, o PostgreSQL exige a dependência `org.flywaydb:flyway-database-postgresql` além de `flyway-core`.
- No perfil `test`, os testes de integração sobem os três bancos com Testcontainers e injetam as conexões com `@ServiceConnection`.

8. Execução

```
cp .env.example .env            # e troque as senhas
docker compose up -d            # PostgreSQL, MongoDB, Redis e a aplicação
docker compose ps               # todos "healthy"
curl localhost:8080/actuator/health
```

- Ordem de subida: os três bancos primeiro; a aplicação só sobe quando os três estão saudáveis (`depends_on` com `service_healthy`).
- Na primeira subida, os scripts de `docker/` criam os usuários. Depois a aplicação aplica as migrações do Flyway e prepara a coleção do MongoDB (D44).
- Para recomeçar do zero: `docker compose down -v`, que apaga os volumes.

O `docker-compose.yml` foi validado com `docker compose config`, e o script de usuários do PostgreSQL foi executado num cluster novo. A subida completa dos contêineres não pôde ser feita aqui, porque o ambiente não alcança o Docker Hub, e o script do MongoDB não foi executado. Os dois são os primeiros itens a conferir na implementação.

9. Verificação

9.1 Esquema do PostgreSQL

`testes-esquema.py`, com 46 casos, está descrito no doc 02, seção 7.4. Os casos U1 a U8 conferem os dois usuários desta seção 3.2.

9.2 Persistência entre os três bancos

`testes-persistencia.py` roda contra PostgreSQL, MongoDB e Redis reais. O entregador do script é uma versão em Python do que a aplicação faz: a lógica é a mesma, então as propriedades testadas valem para ela.

```
pip install "psycopg[binary]" pymongo redis
python3 docs/verificacao/testes-persistencia.py \
  --pg "host=localhost dbname=continuum_teste_esquema user=continuum_app password=teste" \
  --mongo mongodb://localhost:27017 --redis redis://localhost:6379/0
```

Casos	O que verificam
MG0	Preparar a coleção duas vezes não falha (D44)
MG1, MG2	O validador aceita um documento válido de cada tipo e recusa 7 inválidos (campo faltando, faixa, campos de outro tipo, tipo e enum inexistentes)
MG3	A listagem por missão usa `ix_ocorrencias_missao`
OB1, OB2	Caminho feliz; falha no caso de uso não deixa nem histórico nem *outbox*
OB3a–OB3c	MongoDB fora: uma tentativa, a linha é adiada e o ciclo para; nada é reenviado antes da hora; entregue quando o MongoDB volta
OB4	Queda entre a gravação no MongoDB e a marcação no PostgreSQL: reentrega sem duplicar
OB5	Documento recusado fica pendente com o erro e não bloqueia a fila
OB6	4 entregadores em paralelo, 60 ocorrências: cada uma entregue exatamente uma vez
RD1–RD3	Leitura do cache com TTL, invalidação forçando recálculo, Redis fora respondendo direto do PostgreSQL

Resultado na versão 1.0 deste documento: 26 de 26.

9.3 Testes de integração da aplicação (§23)

Os cenários acima viram testes JUnit com Testcontainers. Acrescente um que grave e leia de volta uma ocorrência de cada tipo pelo `MongoTemplate`, para conferir o mapeamento da seção 4.3 (discriminador `tipo` e `Duration`).

10. Decisões de modelagem

D40 — *Outbox* para gravar ocorrências (fecha A14)

A ocorrência vai para o MongoDB pelo *outbox* (seção 6). A transação do PostgreSQL decide sozinha se a ocorrência existe; o MongoDB a recebe depois, de forma idempotente.

- A entrega é **pelo menos uma vez**, e a gravação pelo `_id` a torna efetivamente única.
- O `_id` é gerado na aplicação, antes da transação, para que o histórico já possa citá-lo.
- *Descartada:* gravar no MongoDB primeiro e no PostgreSQL depois, com um job de reconciliação. Deixa uma janela de inconsistência e exige compensar (apagar no MongoDB) quando o PostgreSQL falha.
- *Descartada:* transações distribuídas (2PC/XA), que o MongoDB e o Spring não suportam de forma simples e acoplariam a disponibilidade dos dois bancos.

D41 — MongoDB para ocorrências, e não `JSONB` no PostgreSQL

O PostgreSQL guardaria as ocorrências numa coluna `JSONB` com índices GIN e validação por `CHECK`, e eliminaria o problema da seção 6. Essa alternativa existe e é boa. O MongoDB foi escolhido porque:

- o §20 exige um banco NoSQL cujo modelo principal não seja chave-valor, e as ocorrências são o dado do sistema que melhor se encaixa no modelo de documento;
- o validador `$jsonSchema` por tipo é mais legível que um `CHECK` sobre `JSONB` com seis ramos;
- as ocorrências não participam de nenhuma regra que exija transação com outras tabelas, exceto o histórico, resolvido pelo *outbox*.

O custo é a consistência eventual descrita na seção 6.4.

D42 — Redis como cache dos indicadores, invalidado depois do *commit*

Cache-aside com validade de 60 s, apagado a cada mudança que afeta as contagens, sem persistência em disco e com falha aberta (seção 5).

- *Descartada*: recalcular a cada requisição, que repetiria contagens sobre várias tabelas a cada atualização do painel.
- *Descartada*: manter os contadores atualizados a cada operação (incremento no Redis). É mais rápido, mas qualquer falha deixaria os contadores errados para sempre; com o recálculo, o valor certo sempre volta.

D43 — Dois usuários no PostgreSQL

O Flyway usa o dono do esquema; a aplicação usa um usuário sem DDL e sem `UPDATE` / `DELETE` nos históricos (seção 3.2). Mesmo com uma falha de programação ou uma injeção de SQL, a aplicação não consegue alterar o histórico nem desligar os *triggers* (casos U1 a U5 da suíte do esquema).

D44 — A aplicação prepara a coleção do MongoDB ao subir

Validador e índices são criados pela aplicação, e não por um script de inicialização do contêiner. Assim, qualquer ambiente fica igual, inclusive os testes com Testcontainers, e uma mudança no validador chega a bancos já existentes com `collMod`.

11. Pontos em aberto

Fechados neste documento

#	Questão	Decisão
A14	Como gravar a ocorrência e o histórico sem perder nenhum	*Outbox* (D40)

Novos

#	Questão	Proposta atual	Fechar em
A15	Por quanto tempo manter as linhas processadas do *outbox*?	7 dias, configurável	Implementação do job de limpeza

12. Histórico de revisões

Versão	Data	Alteração
1.0	2026-11-08	Versão inicial: três mecanismos com as respostas do §20, *outbox* com espera crescente (corrigido na revisão), validador do MongoDB, cache no Redis, dois usuários no PostgreSQL, configuração e execução; D40–D44; fecha A14